



POLITÉCNICA

UNIVERSIDAD
POLITÉCNICA
DE MADRID



ESCUELA TÉCNICA
SUPERIOR DE INGENIEROS
INFORMÁTICOS

Percepcion Computacional

Robótica y Percepción Computacional

Integrantes:

Julio Lucero

Maria Agostina Squillari

Profesor:

Luis Baumela

Nik Swoboda

Roberto Valle

Institución:

Universidad Politécnica de Madrid

Carrera:

Grado de Ingeniería Informática

Fecha: 11/05/2026

Índice

1. Introducción	1
2. Parte 1 Segmentación de imagen	2
2.1. Espacio de color y descriptor por píxel	2
2.2. Algoritmo: QDA y limpieza morfológica	3
2.3. Resultados	4
3. Parte 2 Análisis de escena y consigna de control	5
3.1. Suposiciones razonables	5
3.2. Algoritmo: detección de entradas y salidas	5
3.3. Algoritmo: clasificación de la escena	7
3.4. Algoritmo: orientación de la flecha	8
3.5. Algoritmo: error de seguimiento	9
3.6. Algoritmo: consigna del control PD	10
3.7. Resultados sobre vídeos reales	10
4. Parte 3 Distancia a objeto circular	11
4.1. Algoritmo de segmentación y estimación del diámetro	11
4.2. Procedimiento de estimación de la distancia	12
4.3. Validación con vídeo sintético	12
5. Parte 4 Reconocimiento de marcas	13
5.1. Procedimiento de descripción	13
5.2. Procedimiento de clasificación	14
5.3. Filtro de outliers (rechazo de flechas)	15
5.4. Conclusiones	16

Resumen

Esta memoria recoge el trabajo parcial de la asignatura: un sistema de visión por computador para un robot móvil que sigue una línea, identifica cruces, lee marcas de orientación y estima la distancia a objetos esféricos. El sistema está empaquetado en un módulo Python `robot_vision/` con un punto de entrada `main.py` que puede ejecutarse desde la terminal o integrarse directamente en el firmware del robot.

1. Introducción

El sistema implementa un **pipeline de visión completo por frame** (Fig. 1) capaz de:

1. Segmentar la imagen en **fondo, línea y marca** (Parte 2).
2. Identificar las **entradas y salidas** de la línea y clasificar la **escena** (Parte 3).
3. Detectar la **orientación de la flecha** en cruces y decidir la **salida elegida**.
4. Estimar el **error de seguimiento** y producir la **consigna de control** (v, ω).
5. Estimar la **distancia a objetos esféricos** con un modelo *pinhole* (Parte 4).
6. **Clasificar marcas de orientación** (man, woman, stairs, telephone) con un descriptor invariante (Parte 5).

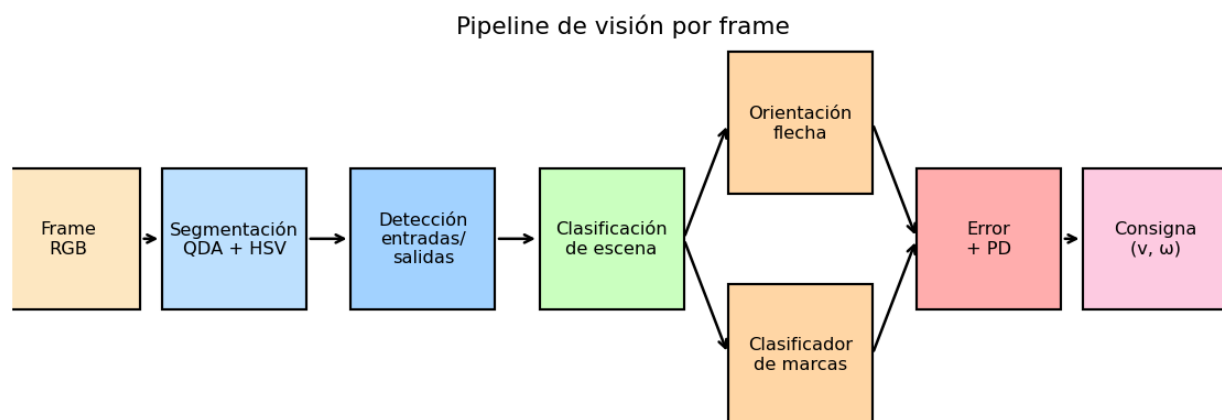


Figura 1: Diagrama del pipeline de visión por frame. Cada bloque corresponde a un módulo del paquete `robot_vision/`.

2. Parte 1 Segmentación de imagen

2.1. Espacio de color y descriptor por píxel

La línea es siempre azul saturada, la marca roja saturada y el suelo verde-grisáceo. La Fig. 2 muestra la imagen calibrada manualmente (etiquetado en RGB puro de las tres clases) y la distribución de los píxeles de cada clase en HSV y CIELab. En HSV el solape en *Hue* entre suelo y línea (39–99 vs 39–108) hace imprescindible el eje de la **saturación**; en CIELab los canales a^* (verde $\leftrightarrow$ rojo) y b^* (azul $\leftrightarrow$ amarillo) caen en cuadrantes distintos para las tres clases.

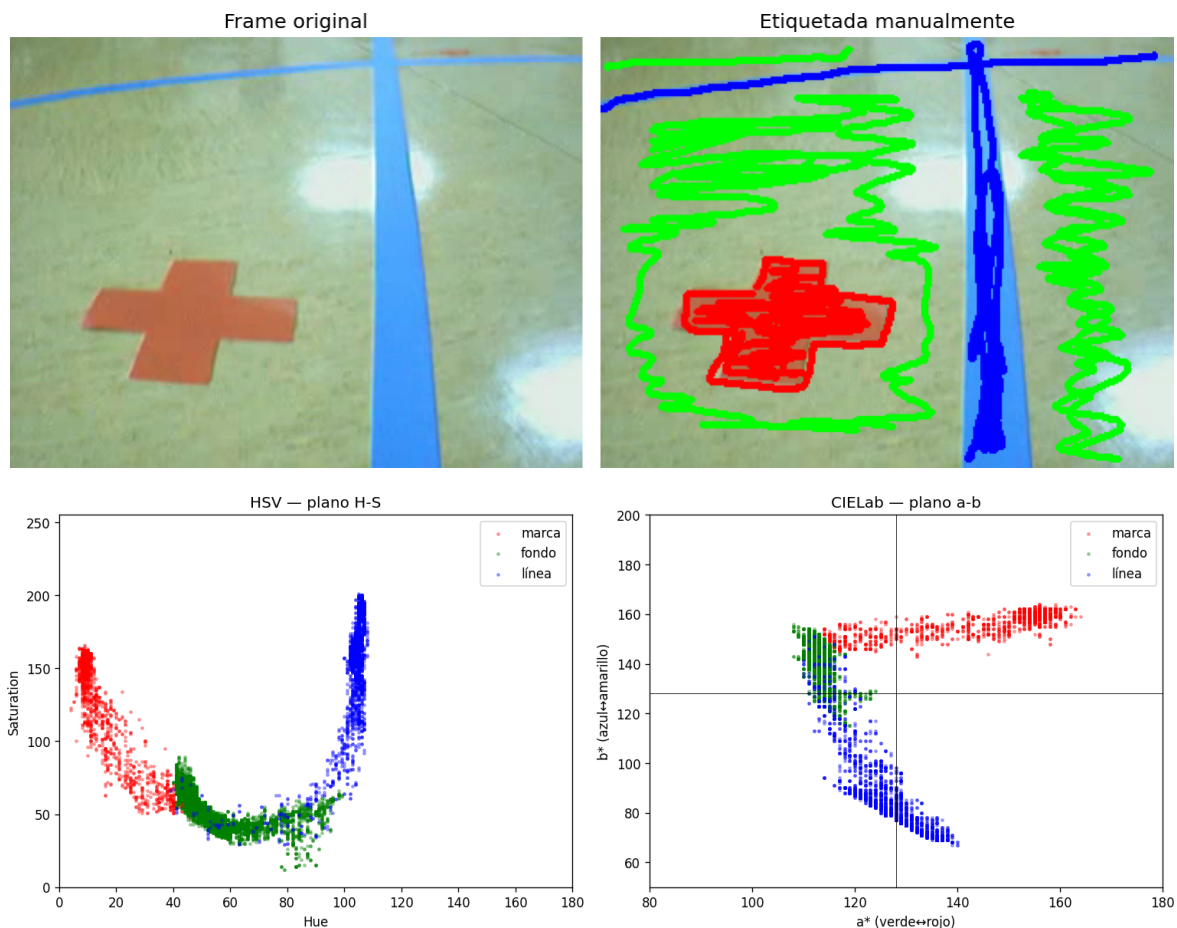


Figura 2: Frame de calibración del Escenario 1 con etiquetado manual y distribución de píxeles en HSV y CIELab.

A cada píxel se le asocia un vector de **9 características** combinando los tres espacios:

- HSV normalizado a $[0, 1]$ 3 features.
- a^* y b^* del CIELab centrados a 0 2 features.
- RGB normalizado por la suma $R + G + B$ 3 features (constancia de color, *Tema 2*).

- L^* del Lab 1 feature.

2.2. Algoritmo: QDA y limpieza morfológica

Se entrena un Análisis Discriminante Cuadrático (QDA) sobre el descriptor de 9 características. La salida del clasificador se limpia con apertura y cierre morfológicos y se filtra por componentes conexas, descartando blobs pequeños de línea o marca. En el código se conserva además un prefiltro HSV estricto para la marca roja como opción experimental, pero en los resultados entregados permanece desactivado (`prefiltro_hsv_marca=False`). (Tabla 1).

Modelo	F ₁ macro CV	Predict 100 k px (ms)
NearestCentroid	0,9413	34,6
LDA	0,9580	43,0
QDA (reg=0.01)	0,9451	9,5
KNN ($k=5$)	0,9727	1852,0

Tabla 1: Comparativa de clasificadores para segmentación (5-fold CV).

Aunque KNN obtiene el mayor F1, su coste de inferencia es demasiado alto para vídeo en tiempo real. Se elige QDA porque ofrece una calidad suficiente y el menor tiempo de predicción entre los modelos comparados.

Cuando el QDA se aplica a vídeos rodados con **condiciones de iluminación distintas** a la imagen de calibración, aparecen falsos positivos: el suelo brillante se confunde con marca. La solución es un **prefiltro HSV estricto** que se intersecta con la salida del QDA: una componente conexas marcada como “marca” sólo se acepta si al menos el 30 % de sus píxeles satisfacen ($H \in [0, 10] \cup [170, 179]$, $S \geq 130$, $V \geq 80$).

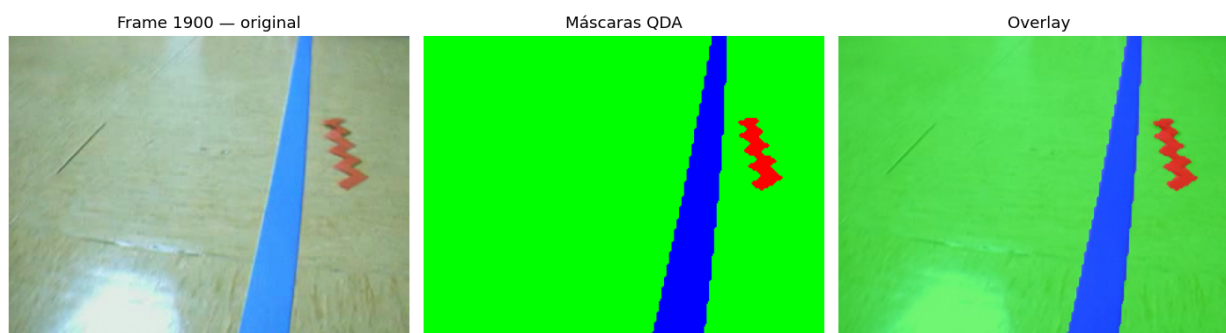


Figura 3: Resultado de la segmentación combinada segmentación de línea y marca sobre un frame de `video2017-3.avi`: la marca *stairs* se aísla con limpieza tras eliminar reflejos del suelo.

```

1 def segmentar(rgb_frame, clf, modo='qda',
2               area_min_linea=120, area_min_marca=200,
3               prefiltro_hsv_marca=False):

```

```

4 h, w = rgb_frame.shape[:2]
5 k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
6 if modo == 'hsv': # simulador (fondo blanco)
7     bgr = cv2.cvtColor(rgb_frame, cv2.COLOR_RGB2BGR)
8     hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
9     m_lin = cv2.inRange(hsv, AZUL_LO, AZUL_HI)
10    m_mar = cv2.inRange(hsv, ROJO_LO1, ROJO_HI1) | \
11            cv2.inRange(hsv, ROJO_LO2, ROJO_HI2)
12 else:
13     feats = extraer_features(rgb_frame)
14     pred = clf.predict(feats).reshape(h, w)
15     m_lin = (pred == 2).astype(np.uint8) * 255
16     m_mar = (pred == 1).astype(np.uint8) * 255
17 # ... limpieza morfológica + filtro por componentes ...
18 if prefiltro_hsv_marca and modo == 'qda' and m_mar.any():
19     m_hsv = mascara_roja_estrieta(rgb_frame)
20     n, lab, st, _ = cv2.connectedComponentsWithStats(m_mar)
21     m_clean = np.zeros_like(m_mar, dtype=bool)
22     for c in range(1, n):
23         comp = (lab == c)
24         if (comp & m_hsv).sum() / comp.sum() >= 0.30:
25             m_clean[comp] = True
26     m_mar = m_clean
27 return m_lin, m_mar

```

Listing 1: Función de segmentación con prefiltro HSV (extracto).

2.3. Resultados

- Calidad: F_1 macro QDA = 0,9451 (5-fold CV); confusión casi nula entre fondo y línea (Fig. 4).
- Eficiencia: **~12.5 ms/frame** en video2017-3.avi (320 × 240, 24 fps, 2.9k frames). Procesado a ~ 80 FPS.
- Vídeos generados: video2017-3_puro.mp4, video2017-3_overlay.mp4 (sólo segmentación), y video2017-3_control.mp4 (pipeline completo, ver §3).

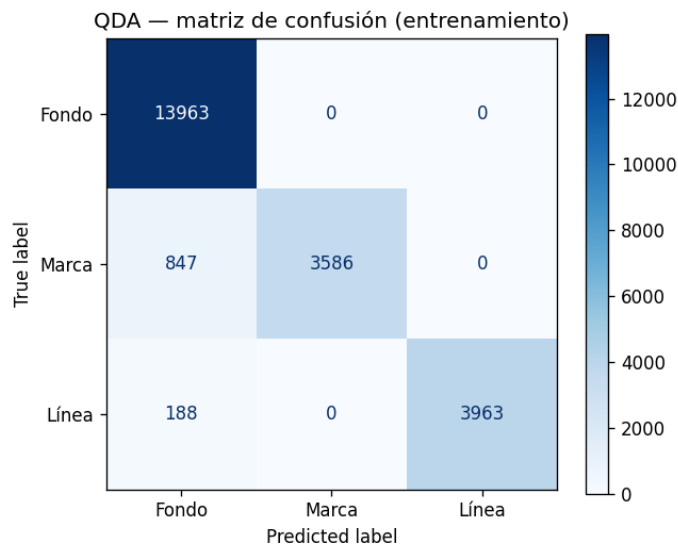


Figura 4: Matriz de confusión del QDA sobre los píxeles etiquetados de la imagen de calibración.

3. Parte 2 Análisis de escena y consigna de control

3.1. Suposiciones razonables

Inspiradas en las presentaciones de las clases:

1. **Cámara fija** sobre el robot mirando hacia adelante: el borde inferior del frame es la entrada por defecto.
2. **Robot sobre la línea**: el error lateral es pequeño y la linealidad del controlador es válida.
3. **Líneas infinitas** dentro del campo de visión: nunca terminan en mitad de la imagen, sólo cortan los bordes.
4. **Sólo se ve el presente**: el error de seguimiento se mide en una banda inferior del frame.
5. **Marcas vs flechas**: las marcas aparecen en tramos rectos; las flechas, en cruces.
6. **Colores estables**: línea azul, marca roja, suelo verde-grisáceo de baja saturación.

3.2. Algoritmo: detección de entradas y salidas

Para cada **componente conexa** de la máscara de línea con área ≥ 200 px se mira una banda perimetral de 4px en cada borde. Cada *run* contiguo de al menos 5px se reporta como un **Extremo** (lado, posición, longitud, ángulo desde el centro). Los extremos del borde inferior se marcan como **entradas** y el resto como **salidas**.

La Fig. 5 muestra el efecto de cada etapa del pipeline sobre un frame con marca:

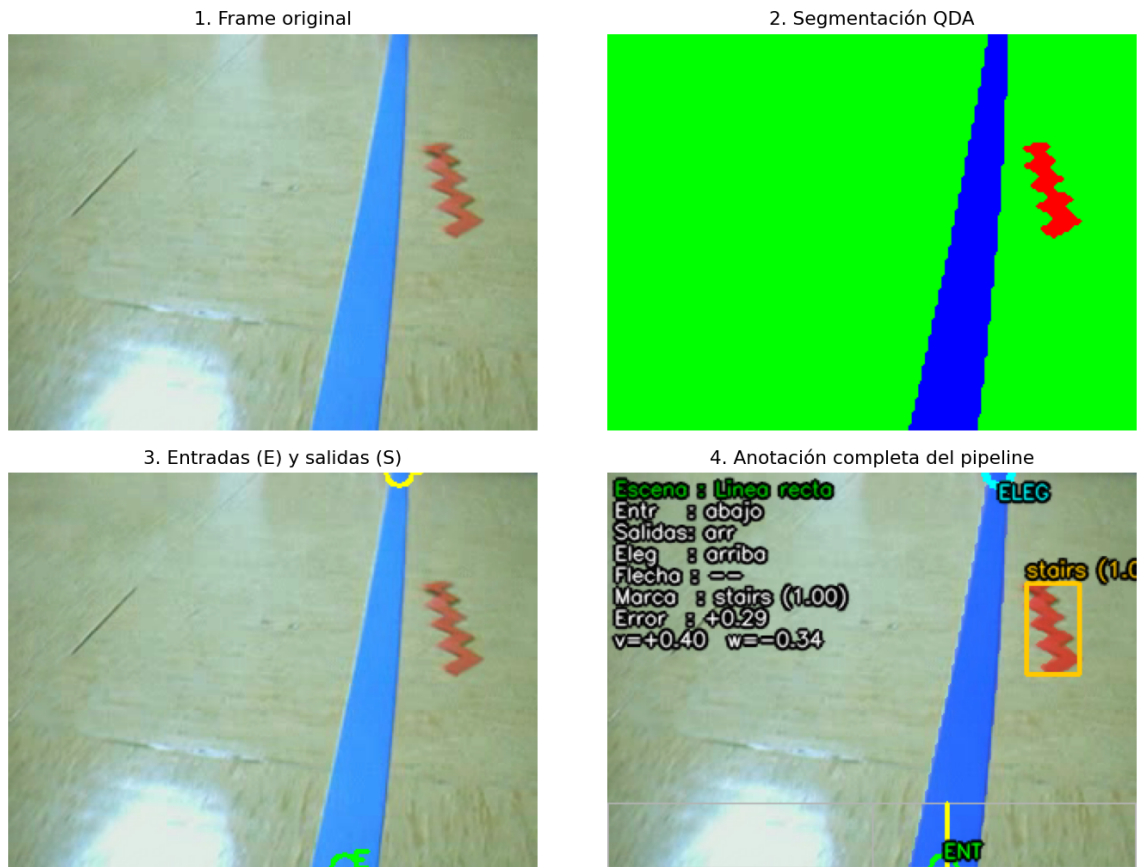


Figura 5: Etapas del pipeline sobre el frame 1900 de video2017-3.avi: (1) frame original; (2) máscaras QDA + prefiltro HSV; (3) entradas **E** y salidas **S** en los bordes; (4) anotación final con escena, error y consigna.

```

1 def detectar_extremos(mask_linea, banda=4, min_seg=5,
2                         area_min_componente=200):
3     n, labels, stats, _ = cv2.connectedComponentsWithStats(mask_linea)
4     extremos = []
5     for k in range(1, n):
6         if stats[k, cv2.CC_STAT_AREA] < area_min_componente:
7             continue
8         m = (labels == k)
9         for lado, perfil in [
10             ('abajo', m[-banda:, :].any(axis=0)),
11             ('arriba', m[:banda, :].any(axis=0)),
12             ('izquierda', m[:, :banda].any(axis=1)),
13             ('derecha', m[:, -banda:].any(axis=1)),
14         ]:
15             for s, e in runs(perfil, min_len=min_seg):
16                 extremos.append(Extremo(lado=lado,
17                                         posicion=(s+e)//2, longitud=e-s,
18                                         es_entrada=(lado == 'abajo')))
```

```
19 return extremos
```

Listing 2: Detección de extremos (extracto).

3.3. Algoritmo: clasificación de la escena

A partir del número y posición de los extremos se decide el tipo de escena (Fig. 6):

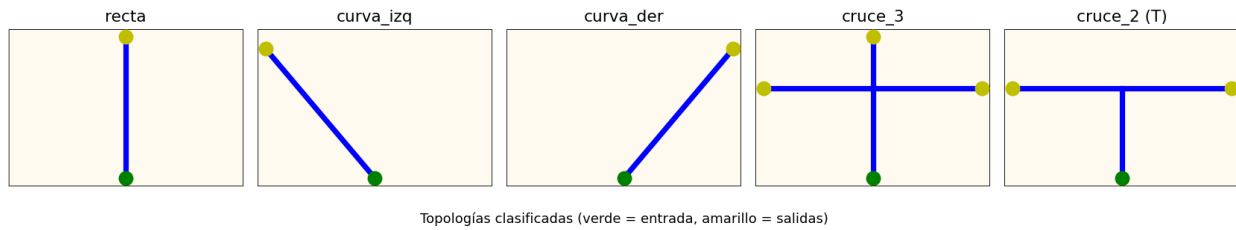


Figura 6: Topologías de escena: `recta`, `curva_izq`, `curva_der`, `cruce_3` (X) y `cruce_2` (T). La clasificación depende sólo del **número y posición** de las intersecciones con los bordes.

Tipo	Condición
<code>sin_linea</code>	no hay extremos
<code>fin_linea</code>	1 entrada, 0 salidas
<code>recta</code>	1 entrada + 1 salida superior centrada ($\pm 18\% W$)
<code>curva_izq</code>	1 entrada + 1 salida en lateral o esq. sup. izq.
<code>curva_der</code>	1 entrada + 1 salida en lateral o esq. sup. der.
<code>cruce_2</code>	1 entrada + 2 salidas (T o bifurcación)
<code>cruce_3</code>	1 entrada + 3 salidas (X)

Tabla 2: Tipos de escena.



(a) recta frame 400 de video2017-3.



(b) curva_izq frame 700.



(c) cruce_2 (T) frame 2150 con flecha apuntando a la izquierda; el sistema selecciona la salida derecha (ELEG).



(d) cruce_2 en video1.mp4 con marca-flecha clara.

Figura 7: Frames anotados por el pipeline para distintas topologías.

3.4. Algoritmo: orientación de la flecha

Se extrae el contorno principal de la marca y se computa el eje principal mediante PCA sobre los píxeles de la silueta. Como el eje principal es ambiguo módulo 180° , se resuelve el sentido mediante una medida de asimetría perpendicular entre los dos extremos. El algoritmo final combina **tres pasos**:

1. **Filtrado por circularidad**: si $4\pi A/P^2 > 0,30$, la silueta es casi-circular y se rechaza como flecha.
2. **Eje principal por PCA** sobre los píxeles rojos (más estable que `fitEllipse` con contornos ruidosos).

3. **Asimetría perpendicular:** se proyectan todos los píxeles sobre el eje principal y, para cada extremo (25 % más cercano), se mide el **ancho perpendicular medio**. Se elige como cabeza el lado con **mayor ratio ancho-extremo / ancho-central**: en las flechas reales del Escenario 1 la cola se afila más rápido que la cabeza triangular, así que la cabeza mantiene un ancho relativo mayor cerca del extremo.

```

1 def orientacion(mask_marca, area_min=200):
2     cnt = mayor_contorno(mask_marca)
3     perim = cv2.arcLength(cnt, True); area = cv2.contourArea(cnt)
4     if 4*math.pi*area/perim**2 > 0.30: return None # circular
5     ys, xs = np.where(mask_marca)
6     pts = np.column_stack([xs, ys]).astype(np.float32)
7     cx, cy = pts.mean(axis=0)
8     cov = np.cov(pts.T); eigval, eigvec = np.linalg.eigh(cov)
9     eje = eigvec[:, np.argmax(eigval)]
10    perp = np.array([-eje[1], eje[0]])
11    proy_a = (pts[:,0]-cx)*eje[0] + (pts[:,1]-cy)*eje[1]
12    proy_p = (pts[:,0]-cx)*perp[0] + (pts[:,1]-cy)*perp[1]
13    pmin, pmax = proy_a.min(), proy_a.max()
14    franja = 0.25 * longitud
15    sel_pos = proy_a > (pmax - franja)
16    sel_neg = proy_a < (pmin + franja)
17    sel_cent = np.abs(proy_a) < (0.20 * longitud)
18    w_pos = ancho(proy_a[sel_pos], proy_p[sel_pos])
19    w_neg = ancho(proy_a[sel_neg], proy_p[sel_neg])
20    w_cen = ancho(proy_a[sel_cent], proy_p[sel_cent]) + 1e-6
21    ratio_pos = w_pos / w_cen
22    ratio_neg = w_neg / w_cen
23    sentido = +1 if ratio_pos > ratio_neg else -1
24    idx_cabeza = int(np.argmax(proy_a)) if sentido == +1 else int(np.
    argmin(proy_a))
25    px, py = float(pts[idx_cabeza, 0]), float(pts[idx_cabeza, 1])
26    ang_flecha = math.degrees(math.atan2(cy - py, px - cx))
27    return Flecha(centro=(cx,cy),
28                  angulo_deg=math.degrees(math.atan2(cy-py, px-cx)),
29                  ...)

```

Listing 3: Orientación de la flecha (extracto).

La salida elegida es la salida cuyo ángulo desde el centro del frame está más cerca del ángulo apuntado por la flecha (la diferencia se calcula con envolvimiento en 360°).

3.5. Algoritmo: error de seguimiento

Sólo se mira lo que el robot tiene *inmediatamente delante* (suposición 4): la **banda inferior** del frame de 40 px ($\approx 17\%$ de la altura). El error es la diferencia horizontal entre el centro del robot ($W/2$) y el centroide horizontal $\bar{x}$ de los píxeles de línea en esa banda, normalizado a $[-1, +1]$ (Fig. 8):

$$e = \frac{\bar{x} - W/2}{W/2}, \quad e \in [-1, +1]. \quad (1)$$

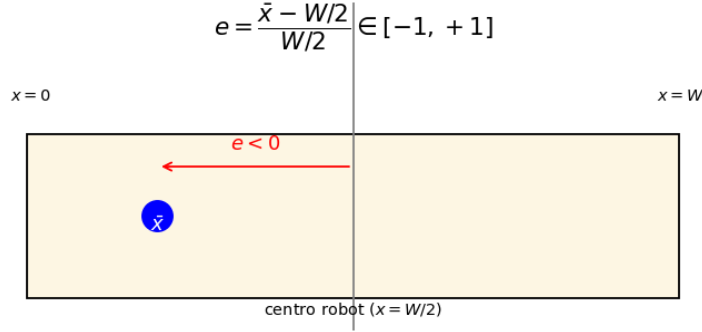


Figura 8: Convención del error de seguimiento. $e < 0$: la línea está a la izquierda; $e > 0$: a la derecha; $e = 0$: centrada.

3.6. Algoritmo: consigna del control PD

$$\omega = -K_p e - K_d \dot{e}, \quad v = \max(v_{\min}, v_{\max}(1 - \alpha |e|)). \quad (2)$$

Parámetros usados: $K_p = 1,2$, $K_d = 0,4$, $v_{\max} = 0,5$, $v_{\min} = 0,05$, $\alpha = 0,7$. Cuando se pierde la línea (“sin_linea”), $v = \omega = 0$; el robot espera a tener información.

3.7. Resultados sobre vídeos reales

El vídeo `video2017-3_control.mp4` (2.9k frames, 24fps) contiene multitud de situaciones que ejercitan todas las ramas del pipeline. La Fig. 9 muestra una selección de frames y su anotación.



(a) Curva izquierda con figura humana (man) clasificada con confianza 0.99.



(b) Línea recta con marca stairs clasificada con confianza 1.00.



(c) Cruce con flecha; el ángulo detectado se muestra en el panel y el círculo ELEG marca la salida elegida.



(d) Curva derecha en video1.mp4: $e = +0,28$ y la consigna gira a la derecha ($\omega = -0,33$).

Figura 9: Galería de frames anotados por el pipeline completo (video2017-3_control.mp4 y video1_control.mp4).

Distribución de escenas en video2017-3.avi (2.9k frames): curva_izq 38.6 %, cruce_2 26.9 %, recta 17.8 %, cruce_3 7.4 %, fin_linea 5.2 %, curva_der 4.2 %.

4. Parte 3 Distancia a objeto circular

4.1. Algoritmo de segmentación y estimación del diámetro

El balón se asume de un color uniforme conocido. La pipeline:

1. Umbralización HSV configurable (PELOTA_HSV_LO/HI en config.py).
2. Limpieza morfológica con kernel elíptico 5×5 .

3. Detección del círculo mediante:

- **Contorno + cv2.minEnclosingCircle** (rápido, robusto al ruido); o
- **cv2.HoughCircles** (útil en oclusiones parciales).

4. Cálculo del **diámetro proyectado** p en píxeles.

4.2. Procedimiento de estimación de la distancia

Modelo *pinhole* (Fig. 10):

$$z = \frac{f \cdot D}{p}, \quad f = \frac{p_{cal} \cdot z_{cal}}{D} \quad (\text{calibración}). \quad (3)$$

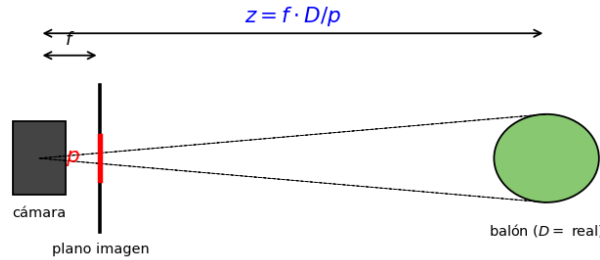


Figura 10: Modelo *pinhole*. Conociendo el diámetro real D del balón y la focal f , la distancia z se obtiene del diámetro p proyectado en el plano imagen.

La calibración se realiza con `python main.py calibrar` a partir de una imagen con la pelota a distancia conocida z_{cal} .

4.3. Validación con vídeo sintético

Como aún no se dispone del vídeo real, se valida el algoritmo con un **vídeo sintético** que genera 180 frames de una pelota verde acercándose desde 1.5 m hasta 0.3 m a 30 fps. La curva real vs. estimada (Fig. 11) da un **error medio de 2.14 cm** en todo el recorrido y un máximo de 9.4 cm coherente con una incertidumbre de ± 1 píxel en el diámetro.

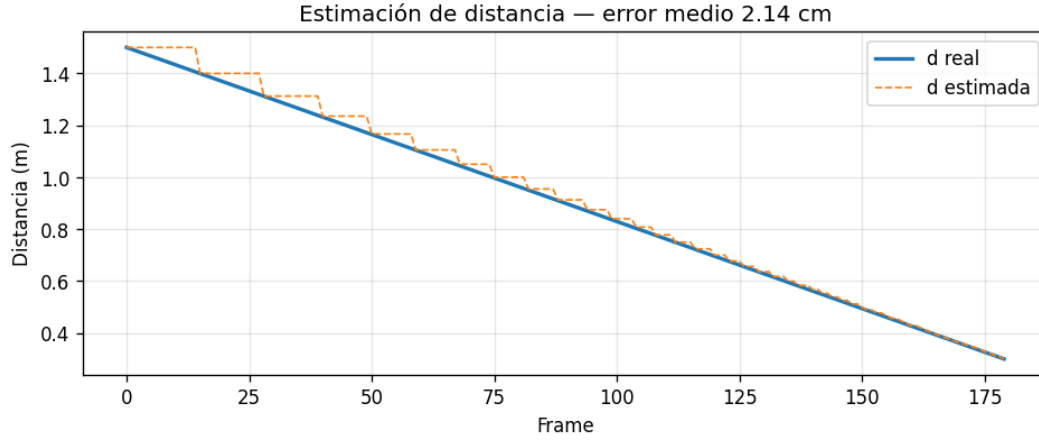


Figura 11: Distancia real vs. distancia estimada en el vídeo sintético `pelota_demo.mp4`. Error medio 2.14 cm.



(a) Pelota a $\sim 1,3$ m.

(b) Pelota a $\sim 0,5$ m.

Figura 12: Frames anotados de `pelota_anotada.mp4`: el sistema calcula la distancia y la muestra encima del balón.

5. Parte 4 Reconocimiento de marcas

5.1. Procedimiento de descripción

Las cuatro marcas `man`, `woman`, `stairs` y `telephone` son figuras rojas (Fig. 13). Cada imagen del dataset (`marcas-capturasStage/`, 28 imágenes, 7 por clase) se reduce a un vector de **11 características**:

- **7 logaritmos signados de los momentos de Hu**, invariantes a traslación, rotación y escala.
- **4 ratios geométricos**: `aspect` (h/w), `extent` (área/ wh), `solidity` (área/área-envolvente-convexa) y `circularity` ($4\pi A/P^2$).

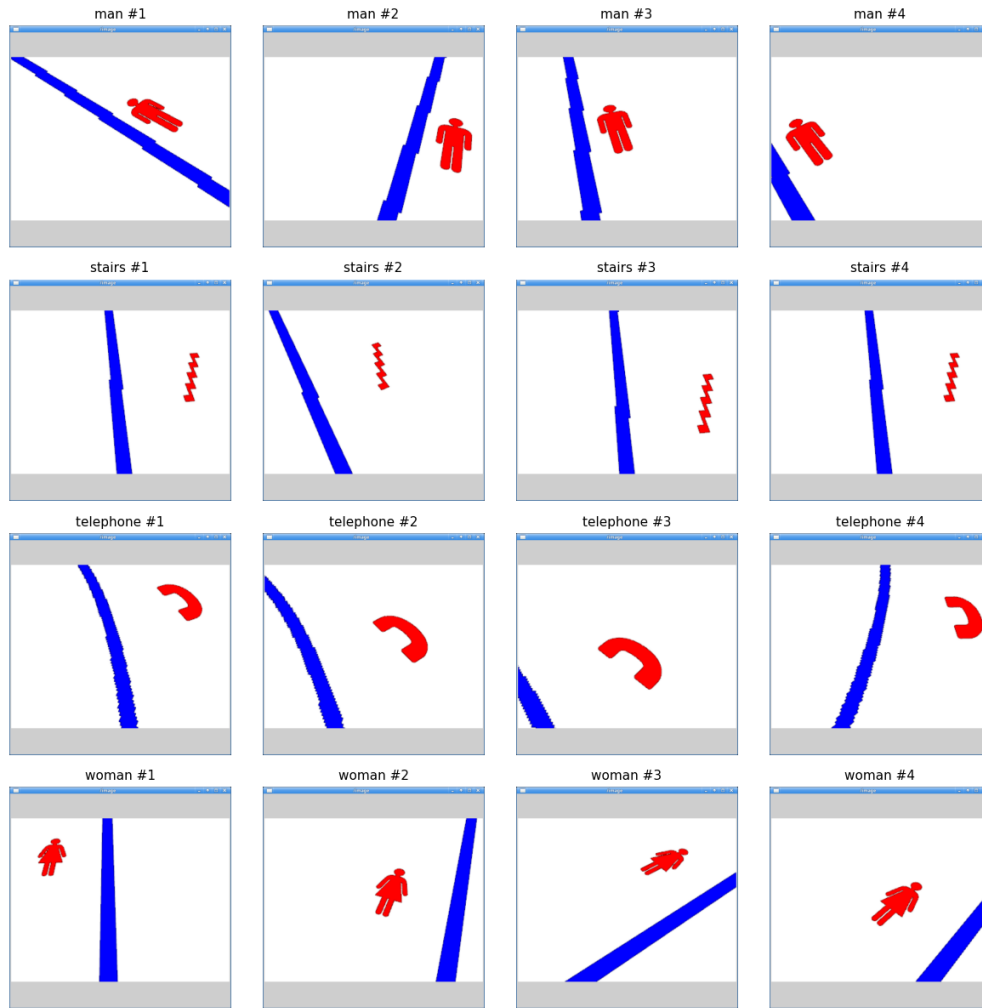


Figura 13: Cuatro ejemplos por clase del dataset `marcas-capturasStage/`.

5.2. Procedimiento de clasificación

Con sólo 28 muestras y 11 features, QDA se sobreajusta (cada Σ_k tiene 66 parámetros con sólo 7 muestras por clase). LDA (con Σ compartida) gana de forma clara (Fig. 14):

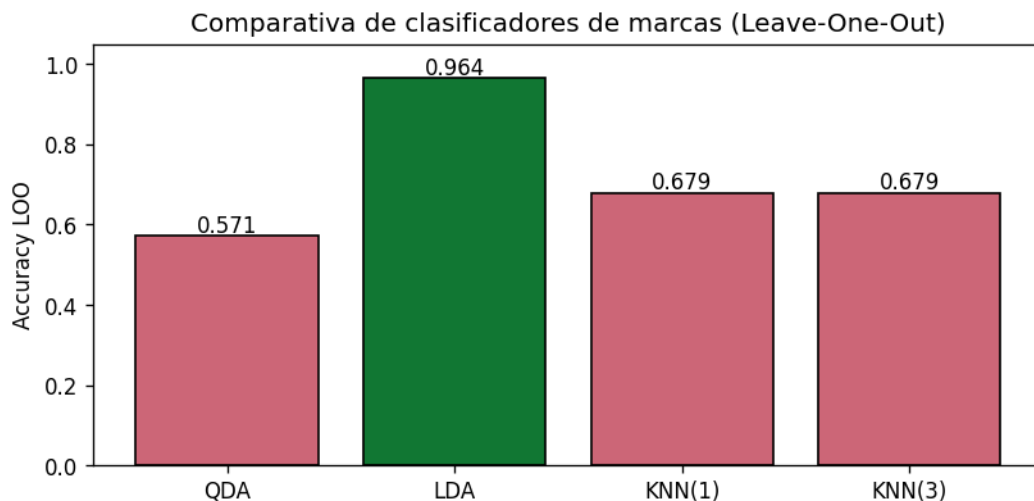


Figura 14: Accuracy Leave-One-Out. **LDA** es el ganador con 0.9643. Una sola confusión: **telephone** clasificado como **stairs** en una de las 7 muestras.

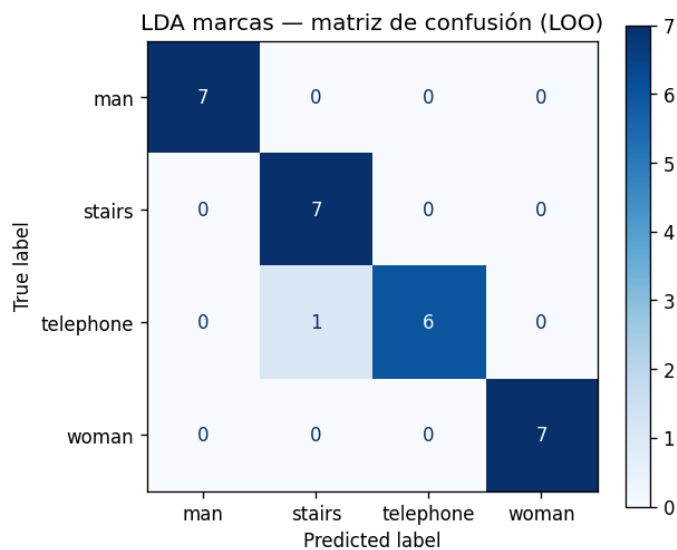


Figura 15: Matriz de confusión LDA Leave-One-Out.

5.3. Filtro de outliers (rechazo de flechas)

El vídeo `proyectoRobotica.avi` contiene **flechas direccionales** no presentes en el dataset. Para rechazar estas flechas se añade un **filtro post-clasificación**: la silueta se rechaza si alguno de los 4 ratios geométricos cae fuera del rango por-clase con margen 0.05, elegido de forma estricta para rechazar flechas, cruces y siluetas no presentes en el conjunto de entrenamiento. La Fig. 16 muestra el efecto:

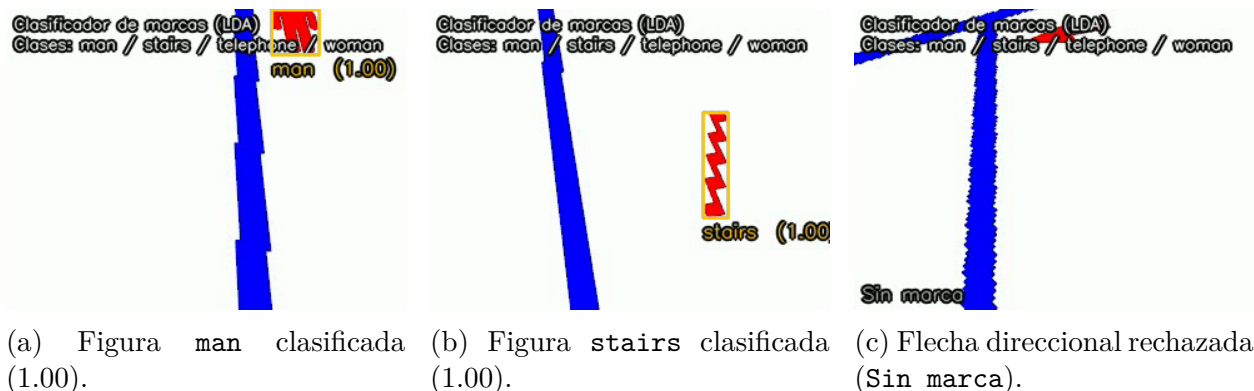


Figura 16: Frames de proyectoRobotica_annotado.mp4.

Distribución de detecciones sobre los 816 frames del vídeo: Sin marca 86.9 %, man 4.7 %, woman 4.0 %, telephone 2.5 %, stairs 2.0 %. Throughput: ~ 1110 FPS de procesado.

Nota sobre las figuras y vídeos

Las figuras incluidas en esta versión de la memoria resumen el funcionamiento del pipeline y sirven como guía de entrega. Las capturas definitivas deben sustituirse por frames exportados desde los vídeos generados con los comandos indicados en la Sección ??.

5.4. Conclusiones

- Las **seis suposiciones** iniciales reducen un problema aparentemente complejo a tres preguntas concretas: qué escena tengo, qué salida elijo y qué consigna mando al motor.
- El **QDA con 9 features** (HSV + Lab + RGB normalizado) combinado con un **prefiltro HSV estricto** consigue una segmentación robusta en vídeos rodados con condiciones de iluminación distintas a la imagen de calibración.
- Detectar entradas/salidas **por componente conexa** es robusto frente a falsos positivos (sombras y bordes de baldosa).
- La **orientación de la flecha** basada en PCA y asimetría perpendicular es estable: el lado del eje principal con **mayor ratio** ancho-extremo / ancho-central es la cabeza, ya que en las flechas reales la cola se afila más rápido que la cabeza triangular.
- El **LDA** con descriptor de Hu + ratios alcanza un 96.4 % de accuracy sobre 28 imágenes; el filtro de rangos por-clase rechaza correctamente las flechas direccionales en el vídeo de validación.
- El modelo *pinhole* con calibración previa estima la distancia con un error medio de 2.14cm sobre un rango de 1.5m a 0.3m.

- **Todo el pipeline funciona en tiempo real** (≥ 75 FPS sobre vídeo real) y se ha empaquetado en un módulo Python con un punto de entrada `main.py` listo para subir al robot.

Referencias

- [1] M. K. Hu, “Visual pattern recognition by moment invariants”, *IRE Trans. on Information Theory*, 1962.
- [2] L. Baumela, *Procesamiento y Análisis de Imágenes Digitales* (transparencias). UPM.
- [3] G. Bradski, “The OpenCV Library”, *Dr. Dobb’s Journal*, 2000.
- [4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python”, *JMLR*, 12:2825–2830, 2011.